

The State of MFEM



Tzanio Kolev
LLNL



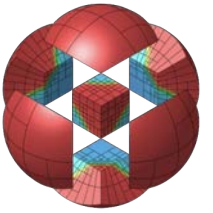
LLNL-PRES-856457

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344. Lawrence Livermore National Security, LLC

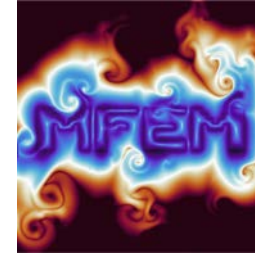


MFEM Finite Element Library

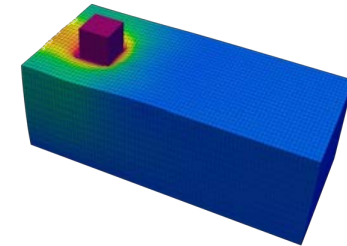
Cutting-edge algorithms for powerful applications on HPC architectures



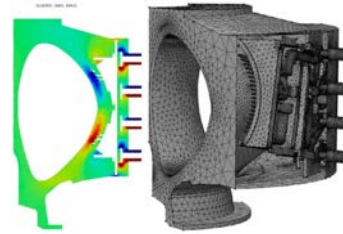
- **Flexible discretizations on unstructured grids**
 - Triangular, quadrilateral, tetrahedral and hexahedral meshes
 - Local conforming and **non-conforming AMR, mesh optimization**
 - Wide variety of finite element methods: Galerkin, DG, IGA, DPG, **HDG**, ...
- **High-order and scalable**
 - Arbitrary-order H1, H(curl), H(div)- and L2 elements
 - Arbitrary order curvilinear meshes
 - MPI scalable to millions of cores and **GPU-accelerated**
 - HPC **differentiable simulations with ∂ FEM** interface
 - Enables application development from laptops to **exascale machines**
- **Built-in solvers and visualization**
 - Integrated with: **HYPRE**, SUNDIALS, PETSc, SLEPc, SUPERLU, ...
 - **AMG preconditioners** for full de Rham complex, geometric MG
 - Support for **GPU solvers** from: HYPRE, PETSc, AmgX, Ginkgo
 - Accurate and flexible visualization with VisIt, ParaView and **GLVis**
- **Open source**
 - Available on GitHub under BSD license, many **example codes + miniapps**



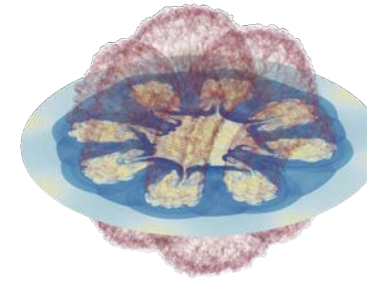
HDG convection-diffusion



Contact mechanics



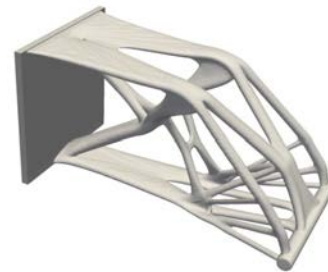
Core-edge tokamak



Compressible flow



Next-gen MRI



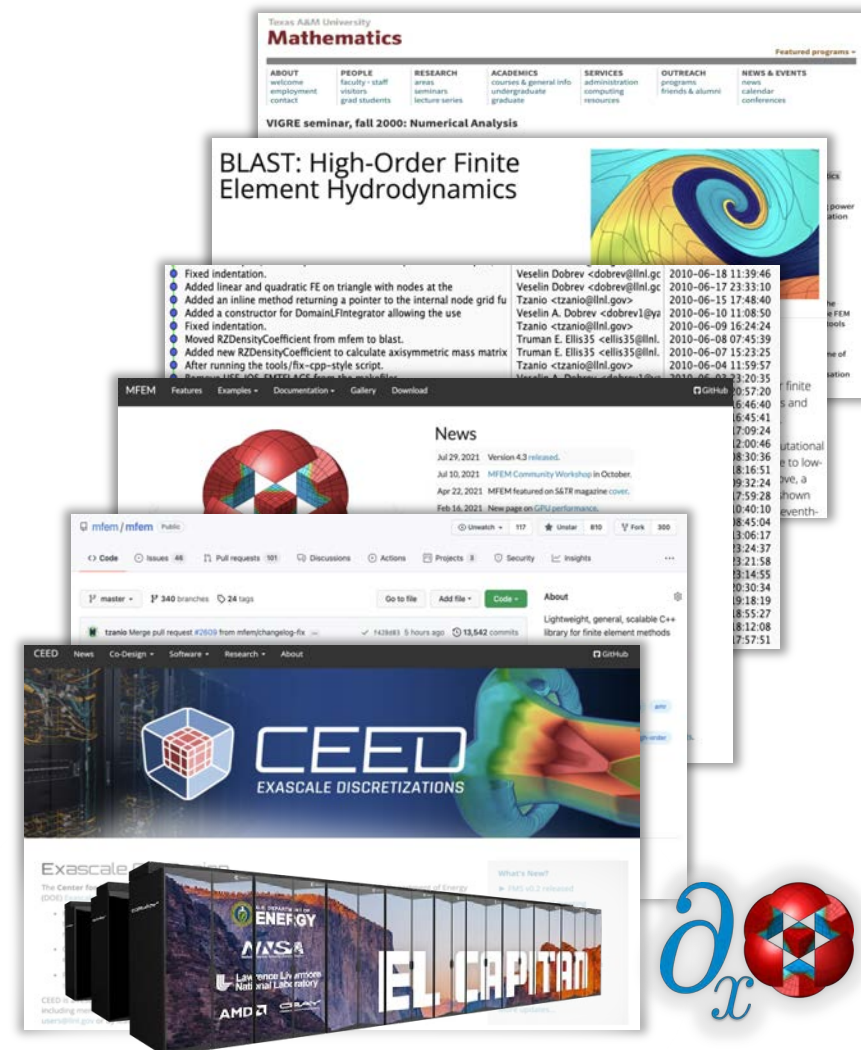
Topology Optimization



A Brief History

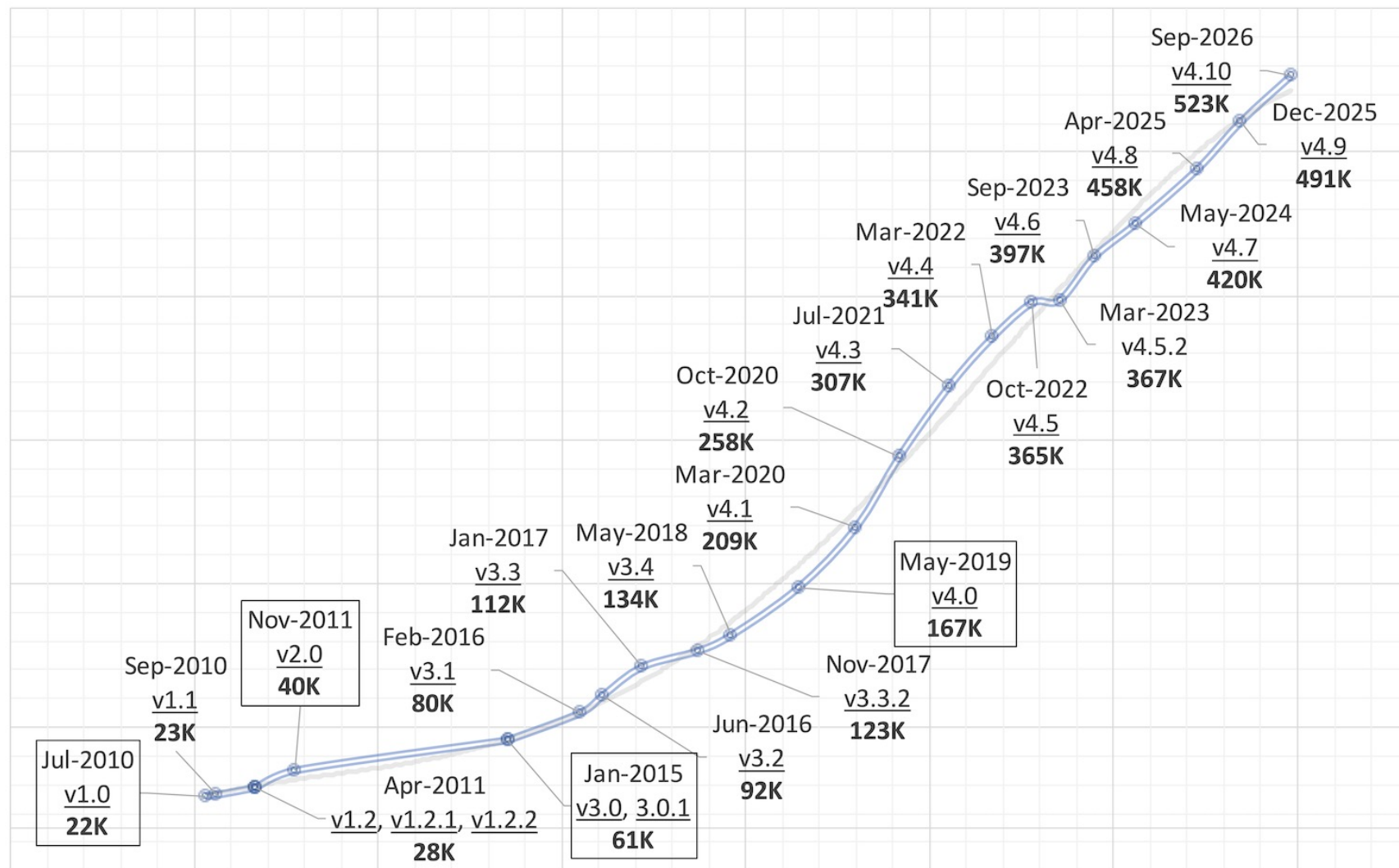
We've been doing this for a long time

- **2000 – “VIGRE seminar: Numerical Analysis” Texas A&M University**
 - Research code: AggieFEM/aFEM
 - Some of the original contributors: [@v-dobrev](#), [@tzanio](#), [@stomov](#)
 - Used in summer internships at LLNL
- **2010 – BLAST project at LLNL**
 - Motivated high-order, non-conforming AMR and parallel scalability developments
 - MFEM repository created in May 2010
 - Some of the original contributors: [@v-dobrev](#), [@tzanio](#), [@rieiben1](#), [@trumanellis](#)
 - Project website [mfem.org](#) goes live in August 2015
- **2017 – Development moved to GitHub**
 - First GitHub commits in February 2017
 - Team expands to include many new developers at LLNL and externally
- **2017 – CEED project in the ECP**
 - Motivated exascale computing developments: GPUs, partial assembly, matrix-free
- **2024 – El Capitan, Differentiable Simulations**



The Source Code is Growing

SLOC in MFEM releases over the last 16 years



mfem-4.10.tgz	v4.10	Sep 2026	4.8M	523K	
mfem-4.9.tgz	v4.9	Dec 2025	4.5M	491K	
mfem-4.8.tgz	v4.8	Apr 2025	4.1M	458K	
mfem-4.7.tgz	v4.7	May 2024	3.8M	420K	
mfem-4.6.tgz	v4.6	Sep 2023	3.6M	397K	
mfem-4.5.2.tgz	v4.5.2	Mar 2023	3.3M	367K	
mfem-4.5.tgz	v4.5	Oct 2022	3.3M	365K	
mfem-4.4.tgz	v4.4	Mar 2022	3.0M	341K	
mfem-4.3.tgz	v4.3	Jul 2021	2.8M	307K	
mfem-4.2.tgz	v4.2	Oct 2020	2.4M	258K	
mfem-4.1.tgz	v4.1	Mar 2020	7.9M	209K	
mfem-4.0.tgz	v4.0	May 2019	5.2M	167K	GPU support
mfem-3.4.tgz	v3.4	May 2018	4.4M	134K	
mfem-3.3.2.tgz	v3.3.2	Nov 2017	4.2M	123K	mesh optimization
mfem-3.3.tgz	v3.3	Jan 2017	4.0M	112K	
mfem-3.2.tgz	v3.2	Jun 2016	3.3M	92K	dynamic AMR, HPC miniapps
mfem-3.1.tgz	v3.1	Feb 2016	2.9M	80K	fem ↔ linear system interface
mfem-3.0.1.tgz	v3.0.1	Jan 2015	1.1M	61K	
mfem-3.0.tgz	v3.0	Jan 2015	1.1M	61K	non-conforming AMR
mfem-2.0.tgz	v2.0	Nov 2011	308K	40K	arbitrary order spaces, NURBS
mfem-v1.2.2.tgz	v1.2.2	Apr 2011	240K	28K	
mfem-v1.2.1.tgz	v1.2.1	Apr 2011	240K	28K	
mfem-v1.2.tgz	v1.2	Apr 2011	240K	28K	MPI parallelism based on hypre
mfem-v1.1.tgz	v1.1	Sep 2010	166K	23K	
mfem-v1.0.tgz	v1.0	Jul 2010	160K	22K	initial release

The Community is Growing

GitHub, downloads, and workshop stats

GitHub

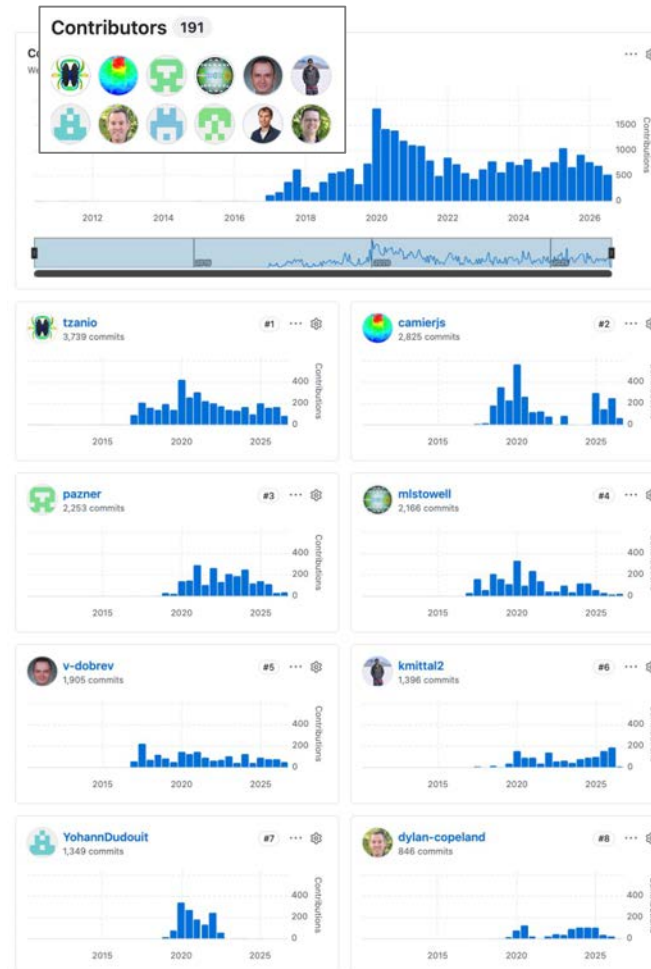
- 191 contributors
- 805 people in the mfem organization – *join to contribute + receive announcements*
- 2249 stars – *thank you!* ★ Starred 2k

Downloads

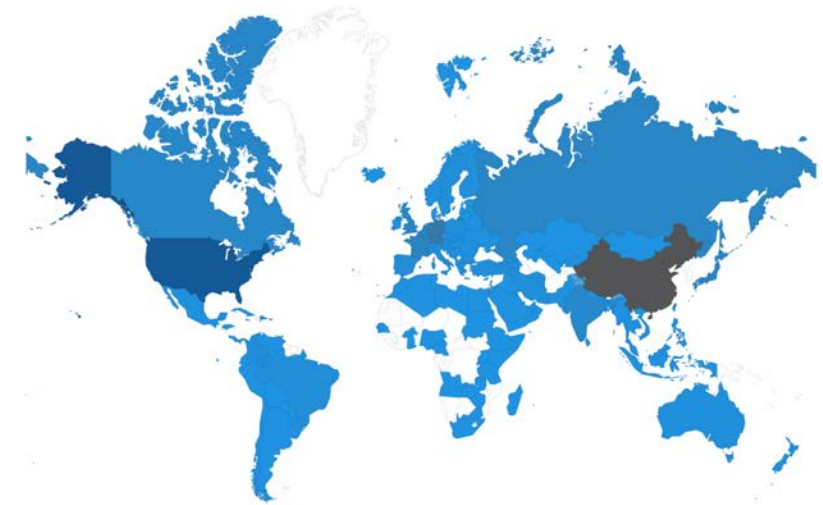
- 150+ unique visitors / day
- 200+ downloads + clones / day
- 139 countries total

2026 Community Workshop

- 290 researchers (90+ in person)
- 30+ postdocs, 100+ students
- 140+ organizations
- 35 countries



Top contributors as of Sep 2026



MFEM has been downloaded from 139 countries

2026 Participants			
289 participants • 145 institutions September 22–25, 2026			
Name	Institution	Email	Attendance
Victor Magri	LLNL	paluettomag1@llnl.gov	Remote
Vivek Singh	CEA	mehic.vivek@gmail.com	Remote
Vladimir Fedorov	Raiden Numerics LLC	Vladimir.Fedorov@raiden-numerics.com	Remote
Vladimir Tomov	LLNL	tomov2@llnl.gov	In person
Wei Zhu	Georgia Tech	weizhu@gatech.edu	In person
WeiJiang Li	Georgia Tech	wli788@oakland.edu	Remote
Wen (Wendy) Zhang	Oakland University	wzhang@oakland.edu	Remote
WenCheng Jin	Texas A&M University	wjincheng_jin@tamu.edu	Remote
Wesley da Silva Pereira	LNCC	wesley.speira@gmail.com	Remote
Will Pamer	Portland State University	pamer@psu.edu	Remote
William Morris	Georgia Tech	wmorris49@gatech.edu	In person
William Orquendo	Universidad Nacional de Colombia	wfquendo@unal.edu.co	Remote
William Pringle	Argonne National Laboratory	wpringle@anl.gov	Remote
William Taitano	Los Alamos National Laboratory	wtaitano@lanl.gov	In person

2026 workshop has 300+ registrations

Latest Releases Were a Team Effort

Versions fem-4.9, mfem-4.10, pymfem-4.10, glvis-4.5

- Released **Dec 2025 / Sep 2026**
- 8 / 9** months in development
- 38 / 48** contributors
- 247 / 191** PRs merged
- 131 / 58** issues closed
- 53K / 48K** new lines of code
- 2456 / 2529** commits
- Many new features:**
 - ∂ FEM initial release
 - AMGF solver + contact miniapp
 - initial support for particle methods
 - improved GPU kernels, TMOP, NURBS,
 - parallel nonuniform anisotropic AMR
 - cuDSS + distributed Ginkgo solvers
 - switch to C++17



All GitHub contributors to mfem up to v4.10

 camierjs 737 commits	 helloworld922 516 commits	 tzanio 507 commits
 kmittal2 438 commits	 v-dobrev 264 commits	 pazner 256 commits
 rzhangbq 147 commits	 j-signorelli 135 commits	 psocratis 127 commits
 dylan-copeland 122 commits	 najlkin 122 commits	 vladotomov 113 commits
 jandrej 110 commits	 mlstowell 92 commits	 IdoAkkerman 83 commits

Top 15 contributors to the latest release

New GLVis release

- glvis-4.5** released in February
- New features:**
 - complex data visualization
 - color palettes enhancements
 - visualization of DOF locations
 - new headless mode
 - code base refactoring
 - glvis-js improvements



New PyMFEM release

- PyMFEM-4.10** released on Monday!
- caught up with mfem-4.10
- MacOS binary packages

Recent Releases

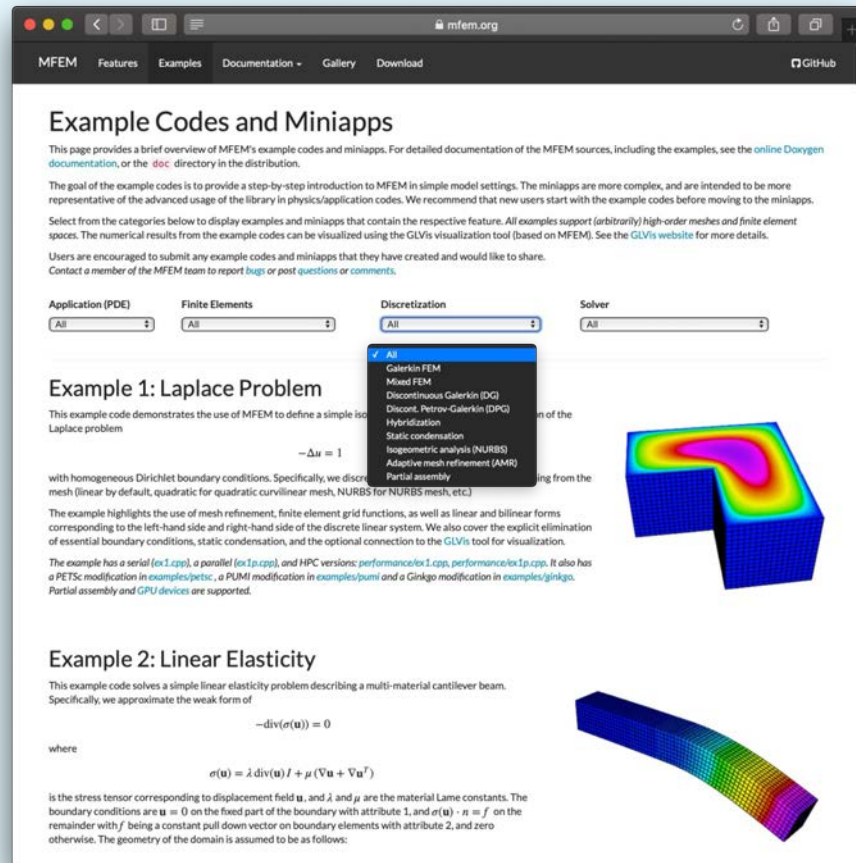
Stars since mfem-4.0 (2019-2026)

Release	Months	Workdays	ΔSLOC K	Files	Additions	Deletions	Contributors	Commits	Issues	Merged	PRs
v4.0	12	249	+33	277	53,803	6,600	36	1,537	200		160
v4.1	10	197	+42	477	105,701	12,381	41	2,644	224		131
v4.2	8	164	+49	528	96,297	25,474	48	3,242	243		198
v4.3	9	185	+49	779	89,800	19,147	54	3,806	262		241
v4.4	8	159	+34	833	75,018	28,241	43	2,207	213		179
v4.5	7	149	+24	461	44,341	7,771	42	1,693	148		129
v4.6	11	233	+32	974	69,712	22,244	39	2,180	243		234
v4.7	7	152	+23	1,138	60,327	25,342	42	1,694	155		166
v4.8	11	230	+38	1,105	70,196	16,715	36	2,922	283		182
v4.9	8	169	+33	625	75,520	20,097	38	2,456	130		153
v4.10	9	181	+32	1,243	64,038	15,992	48	2,529	59		191
Average	9.1	188.0	+35.4	767.3	73,159.4	18,182.2	42.5	2,446.4	196.4		178.5

- mfem-4.9 and mfem-4.10 are fairly typical
- except for the number of issues: we see a decline, because less issues were opened in the first place
- we don't see such a drop in the PR stats

Examples

The first stop for new users



Example Codes and Miniapps

This page provides a brief overview of MFEM's example codes and miniapps. For detailed documentation of the MFEM sources, including the examples, see the [online Doxygen documentation](#), or the [doc](#) directory in the distribution.

The goal of the example codes is to provide a step-by-step introduction to MFEM in simple model settings. The miniapps are more complex, and are intended to be more representative of the advanced usage of the library in physics/application codes. We recommend that new users start with the example codes before moving to the miniapps.

Select from the categories below to display examples and miniapps that contain the respective feature. All examples support (arbitrarily) high-order meshes and finite element spaces. The numerical results from the example codes can be visualized using the GLVis visualization tool (based on MFEM). See the [GLVis website](#) for more details.

Users are encouraged to submit any example codes and miniapps that they have created and would like to share. Contact a member of the MFEM team to report [bugs](#) or post [questions](#) or [comments](#).

Application (PDE): All | Finite Elements: All | Discretization: All | Solver: All

Example 1: Laplace Problem

This example code demonstrates the use of MFEM to define a simple Laplace problem

$-\Delta u = 1$

with homogeneous Dirichlet boundary conditions. Specifically, we discretize the problem using a mesh (linear by default, quadratic for quadratic curvilinear mesh, NURBS for NURBS mesh, etc.)

The example highlights the use of mesh refinement, finite element grid functions, as well as linear and bilinear forms corresponding to the left-hand side and right-hand side of the discrete linear system. We also cover the explicit elimination of essential boundary conditions, static condensation, and the optional connection to the GLVis tool for visualization.

The example has a serial ([ex1.cpp](#)), a parallel ([ex1p.cpp](#)), and HPC versions: [performance/ex1.cpp](#), [performance/ex1p.cpp](#). It also has a PETSc modification in [examples/petsc](#), a PUMI modification in [examples/pumi](#) and a Ginkgo modification in [examples/ginkgo](#). Partial assembly and GPU devices are supported.

Example 2: Linear Elasticity

This example code solves a simple linear elasticity problem describing a multi-material cantilever beam. Specifically, we approximate the weak form of

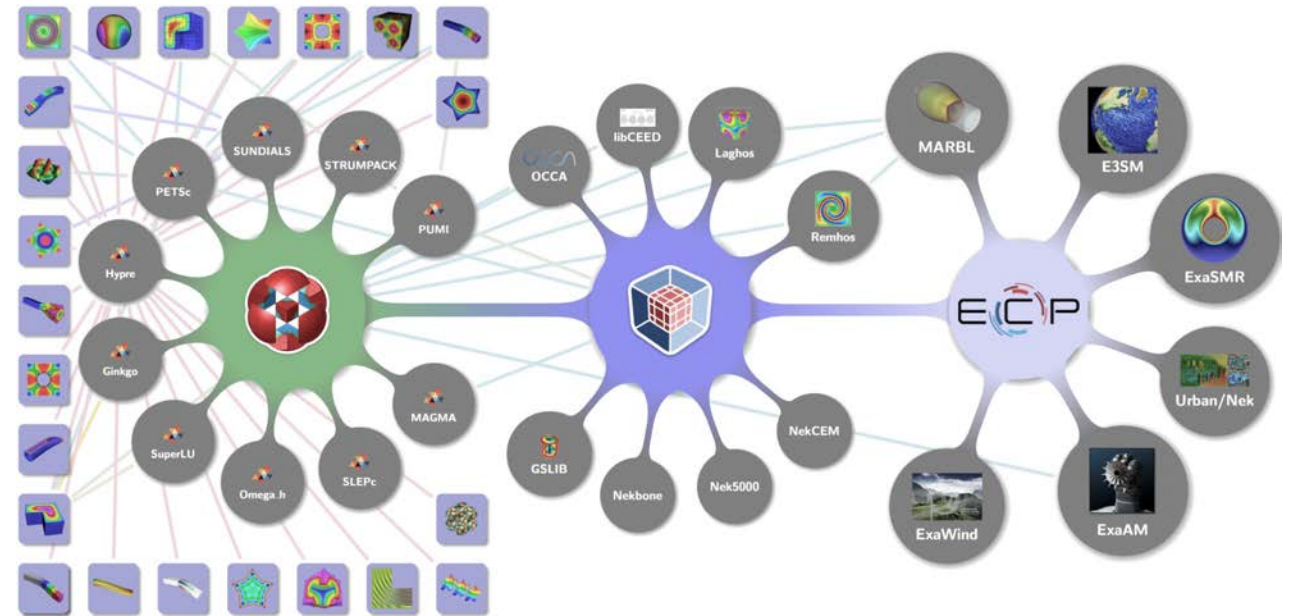
$$-\operatorname{div}(\sigma(\mathbf{u})) = \mathbf{0}$$

where

$$\sigma(\mathbf{u}) = \lambda \operatorname{div}(\mathbf{u}) \mathbf{I} + \mu (\nabla \mathbf{u} + \nabla \mathbf{u}^T)$$

is the stress tensor corresponding to displacement field $\mathbf{u}$, and λ and μ are the material Lamé constants. The boundary conditions are $\mathbf{u} = \mathbf{0}$ on the fixed part of the boundary with attribute 1, and $\sigma(\mathbf{u}) \cdot \mathbf{n} = \mathbf{f}$ on the remainder with $\mathbf{f}$ being a constant pull down vector on boundary elements with attribute 2, and zero otherwise. The geometry of the domain is assumed to be as follows:

mfem.org/examples



- 40+ example codes, most with both serial + parallel versions
- Tutorials to learn MFEM features
- Starting point for new applications
- Show integration with many external packages, miniapps

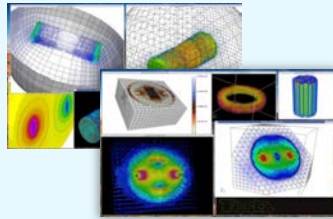
Miniapps

More advanced, ready-to-use physics solvers

Volta, Tesla, Maxwell and Joule Miniapps

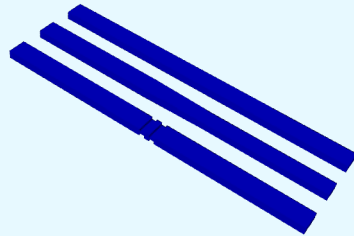
Static and transient electromagnetics

- **Volta** $-\nabla \cdot \epsilon \nabla \varphi = \rho - \nabla \cdot \vec{P}$
- **Tesla** $\nabla \times \mu^{-1} \nabla \times \vec{A} = \vec{J} + \nabla \times \mu^{-1} \mu_0 \vec{M}$

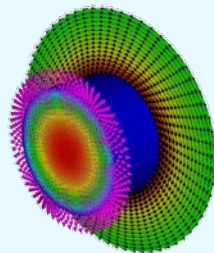


- **Maxwell** · *transient full-wave EM*

$$\frac{\partial(\epsilon \vec{E})}{\partial t} = \nabla \times (\mu^{-1} \vec{B}) - \sigma \vec{E} - \vec{J}$$
$$\frac{\partial \vec{B}}{\partial t} = -\nabla \times \vec{E}$$



- **Joule** · *transient magnetics + Joule heating*
- Arbitrary order elements + meshes
- Adaptive mesh refinement



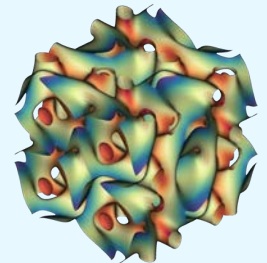
mfem.org/electromagnetics

Navier Miniapp

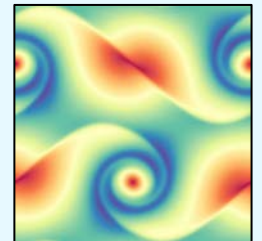
Transient incompressible Navier-Stokes equations

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} - \nu \Delta \mathbf{u} + \nabla p = \mathbf{f}$$
$$\nabla \cdot \mathbf{u} = 0$$

- Arbitrary order elements
- Arbitrary order curvilinear mesh elements
- Adaptive IMEX (BDF-AB) time-stepping algorithm up to 3rd order
- State-of-the-art HPC performance
- GPU acceleration
- Convenient user interface



3D Taylor-Green vortex, 7th order

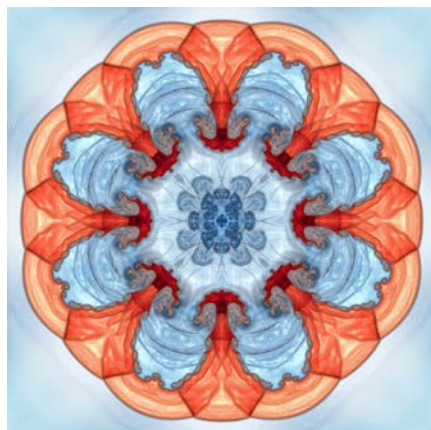


Double shear layer, 5th order, Re = 100000

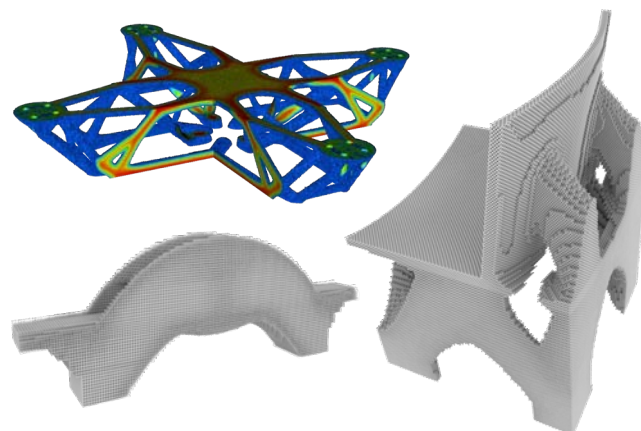
mfem.org/fluids

Applications

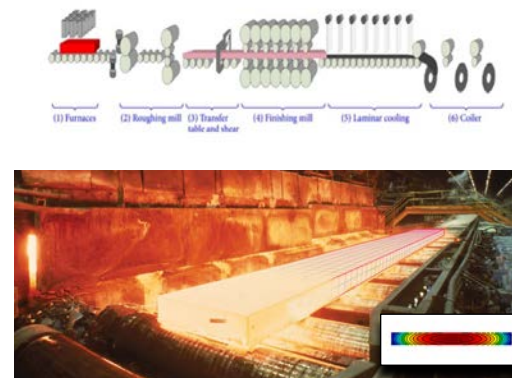
Some of the large-scale simulation codes powered by MFEM



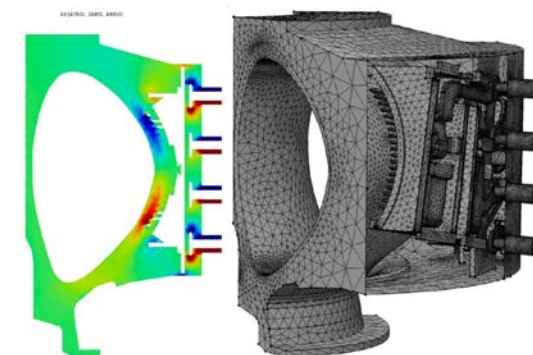
Inertial confinement fusion (BLAST, LLNL)



Topology optimization for additive manufacturing (LiDO, LLNL)



Hot strip mill slab modeling (U.S. Steel)



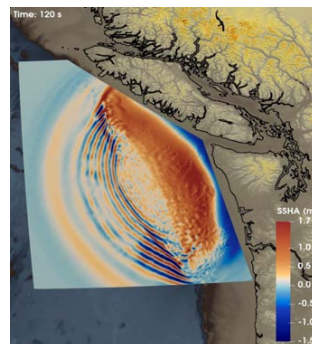
Core-edge tokamak EM wave propagation (SciDAC, PPPL)



Electric aircraft design (RPI)



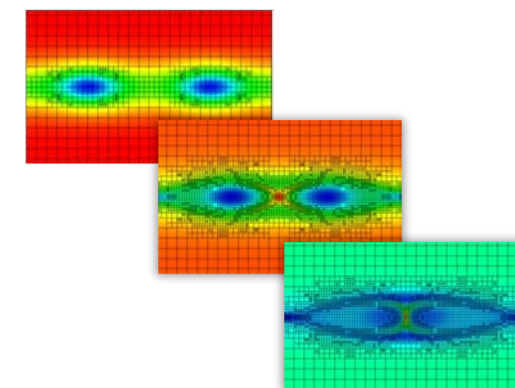
MRI modeling (Harvard Medical)



Tsunami warning (Cascadia, UT/UCSD)



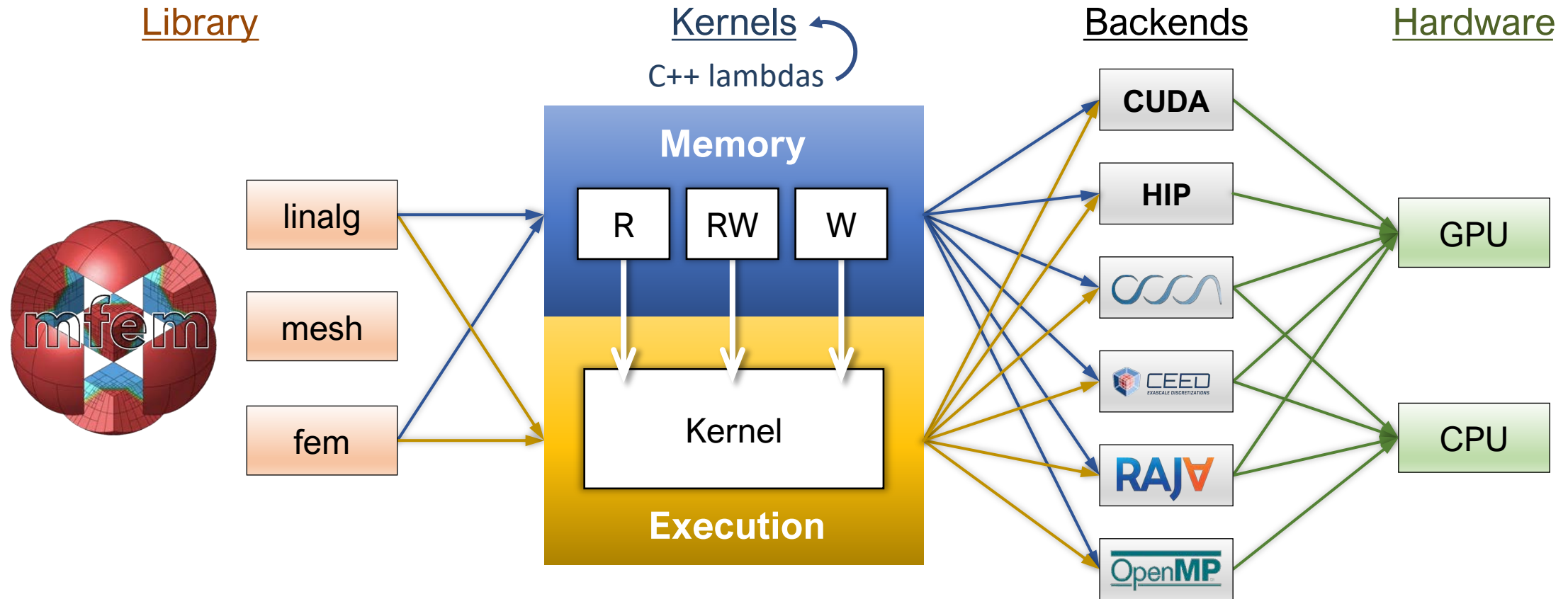
Quantum Computing Hardware (Palace, Amazon)



Adaptive MHD island coalescence (SciDAC, LANL)

GPU Support as a First-class Citizen

MFEM has provided GPU acceleration for over 7 years (since mfem-4.0)

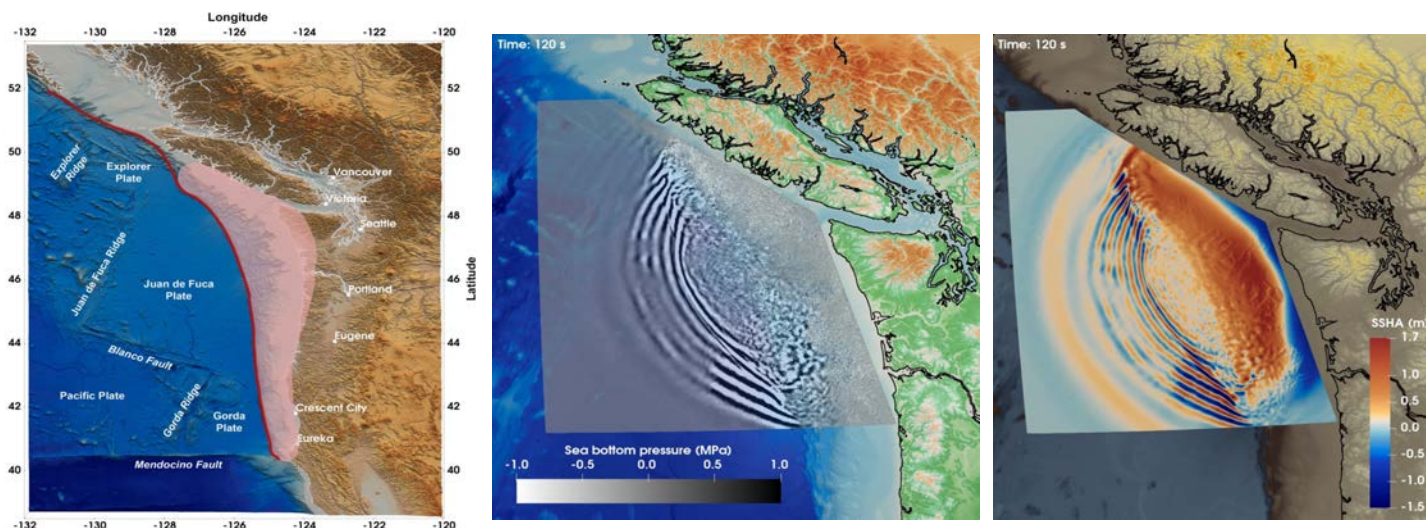


- Matrix-free partial assembly (PA)
- runtime-selectable backends
- ready for future hardware

Real-Time Tsunami Early Warning

MFEM-bases acoustic-gravity simulations on El Capitan

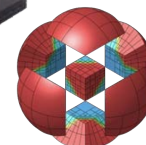
S. Henneking, S. Venkat, M. Fernando, O. Ghattas (UT Austin)
V. Dobrev, J. Camier, Tz. Kolev (LLNL), A. Gabriel (UCSD)



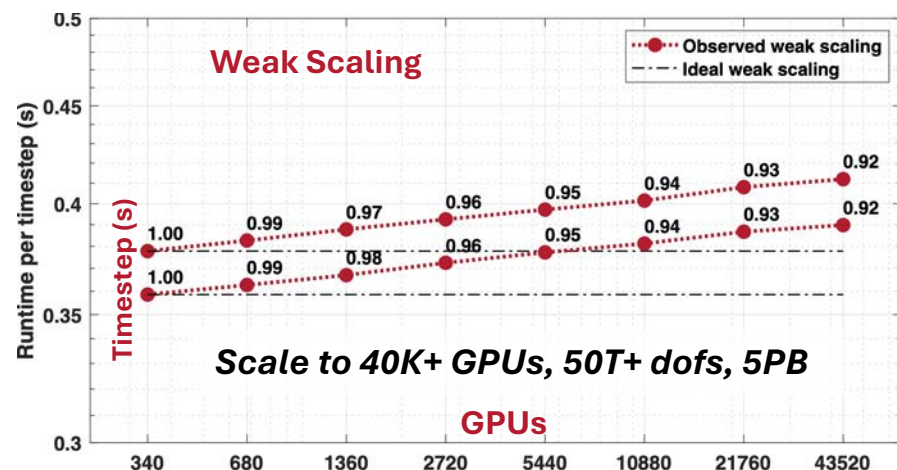
Left: locked part of the plates at the Cascadia subduction zone (CSZ); Center/Right: sea bottom pressure/sea surface wave height for a realistic partial rupture scenario computed with the acoustic-gravity PDE model implemented in MFEM.



$$\begin{aligned} \rho \partial \mathbf{u} / \partial t + \nabla p &= 0 & \text{in } \Omega \times (0, T) \\ K^{-1} \partial p / \partial t + \nabla \cdot \mathbf{u} &= 0 & \text{in } \Omega \times (0, T) \\ p &= \rho g \eta, \quad \partial \eta / \partial t = \mathbf{u} \cdot \mathbf{n} & \text{on } \Gamma_{\text{surface}} \times (0, T) \\ \mathbf{u} \cdot \mathbf{n} &= -\partial b / \partial t & \text{on } \Gamma_{\text{bottom}} \times (0, T) \\ \mathbf{u} \cdot \mathbf{n} &= p / \rho c & \text{on } \Gamma_{\text{absorb}} \times (0, T) \end{aligned}$$



Mixed formulation for pressure, velocity and wave height, 4th order elements, RK4.



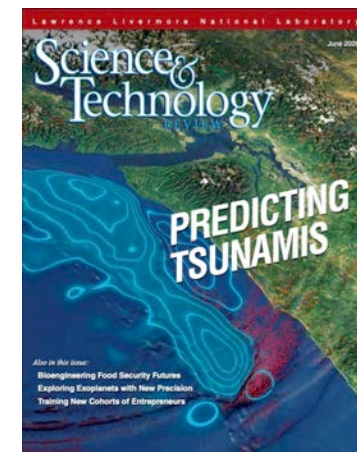
Weak Scaling:
unknowns per GPU

1,275,068,416
(upper)

1,207,959,552
(lower)

Largest Total

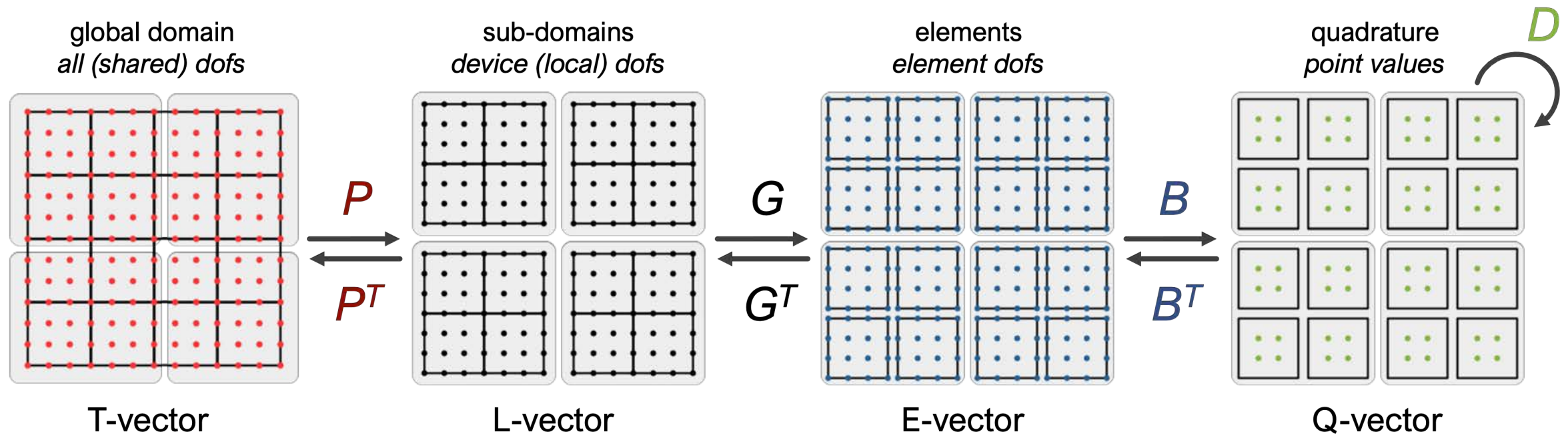
55.5T unknowns
4.9 PB



FEM Operator Decomposition + Partial Assembly for HPC

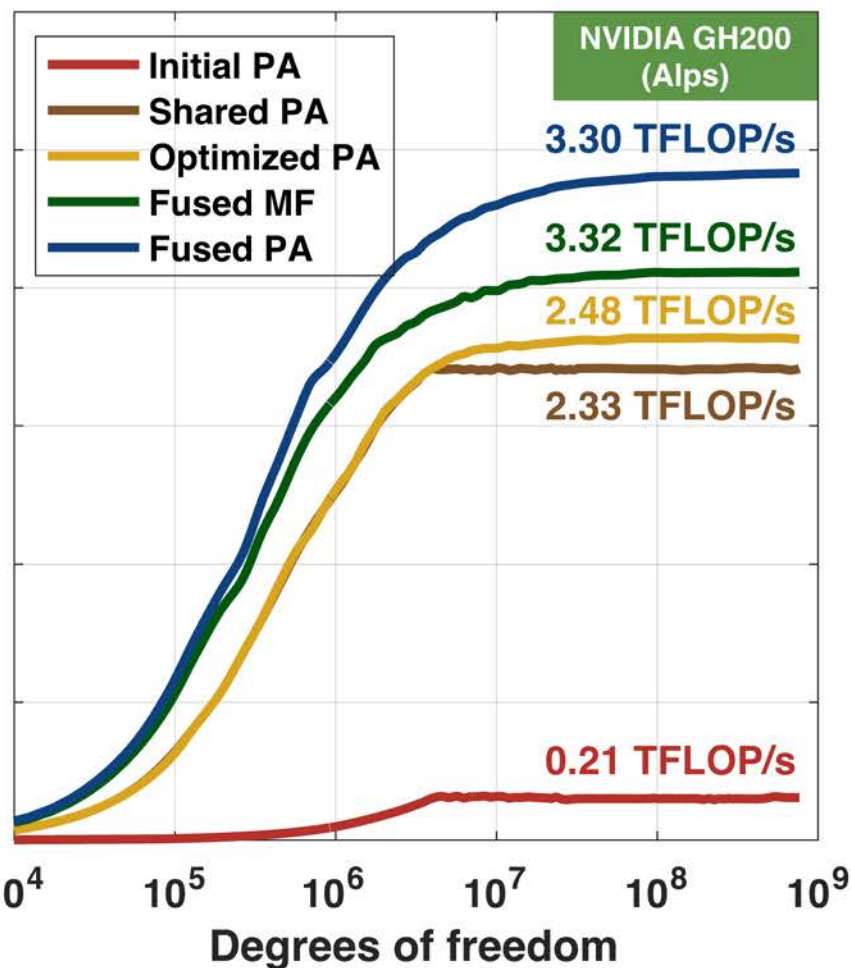
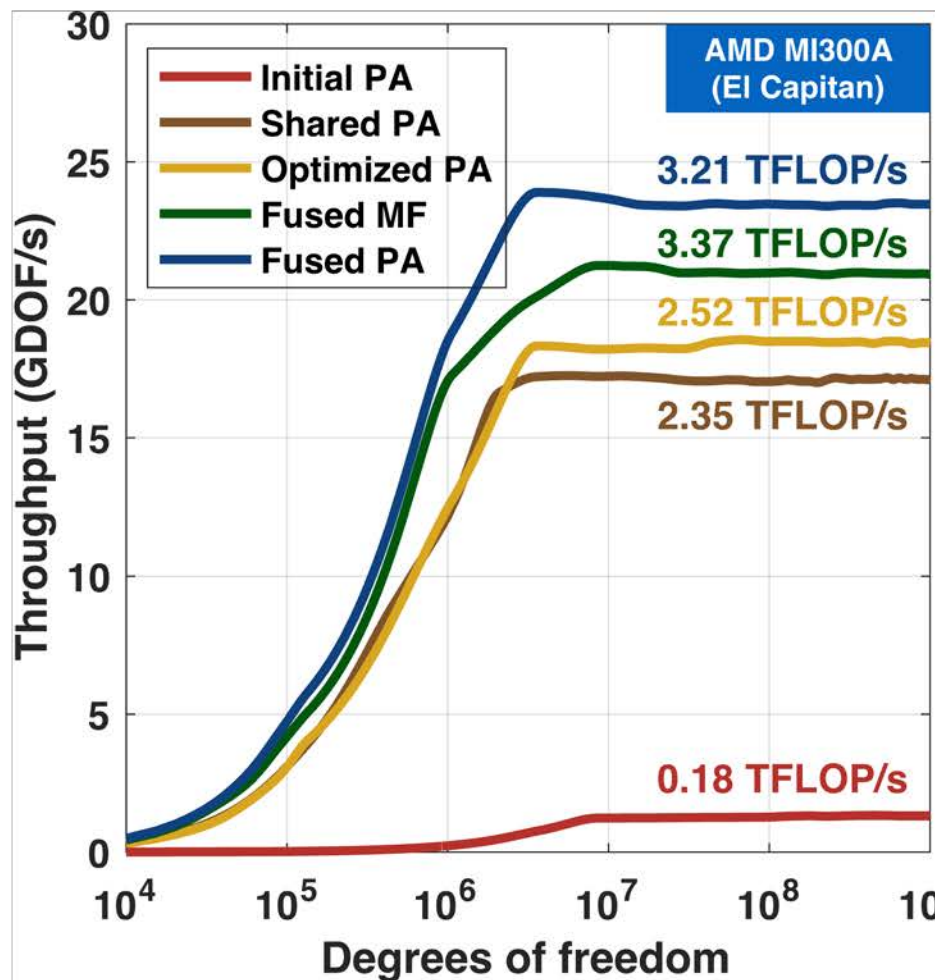
Decompose A into parallel, mesh, basis, and geometry/physics parts

$$A = P^T G^T B^T D B G P$$



- Partial assembly = store only D , evaluate B 
- Optimal memory, near-optimal FLOPs compared to A
- Enabled AMR, High-Order methods, GPUs
- Enables Differentiable Simulations

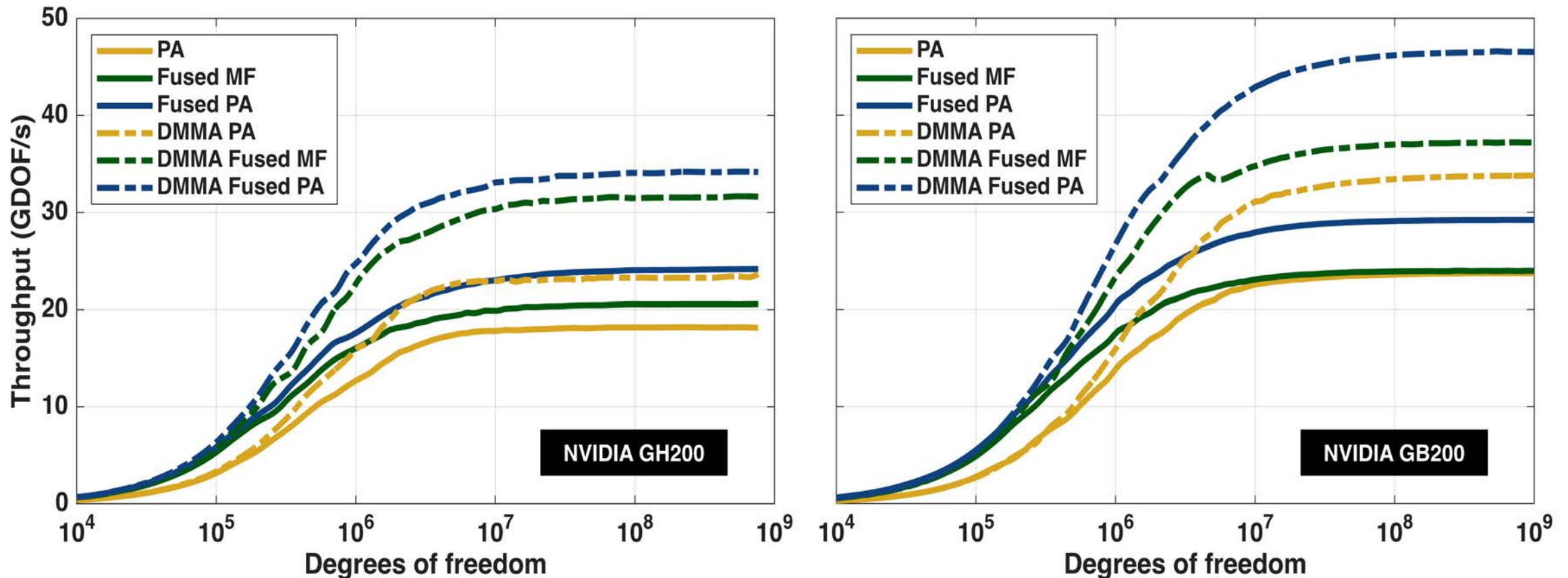
GPU kernel optimizations for Cascadia on El Capitan and Alps



- **Fused PA**
Fuses operators into single kernel
- **Fused matrix-free MF**
Computes entirely matrix-free
- **Optimized PA**
Uses explicit launch bounds
- **Shared PA**
Exploits GPU shared-memory
- **Partial assembly (PA)**
Stores data at quadrature points

Additional improvements with FP64 tensor cores

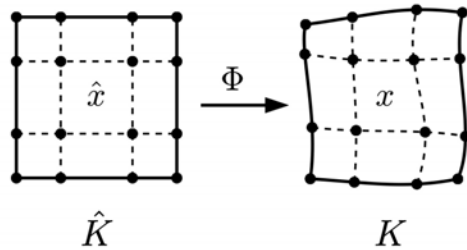
Tensor-core-enabled **Double-precision Matrix-Multiply-Accumulate** (DMMA) contractions with MFEM on Grace-Hopper and Grace-Blackwell Superchips



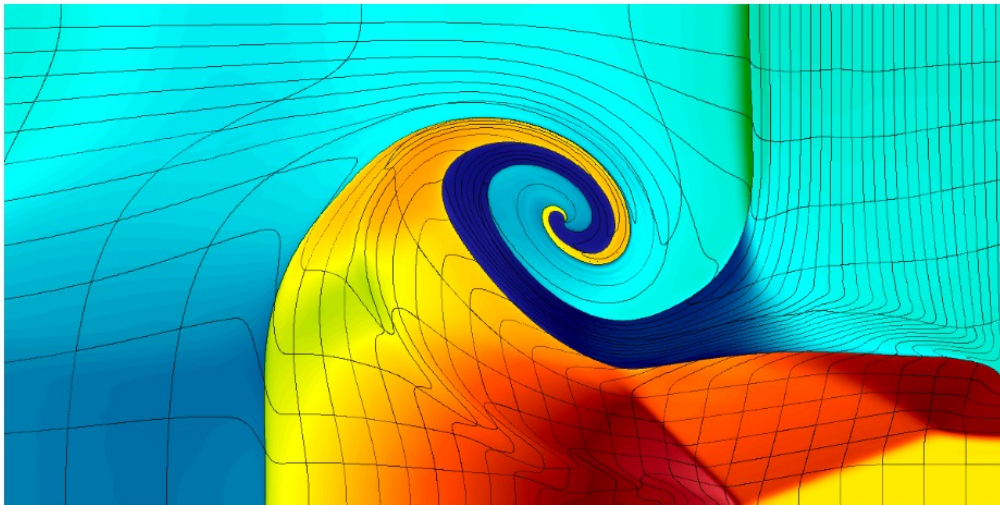
High-Order Finite Elements on High-Order Meshes

Nodal basis for high-order spaces in the de Rham complex

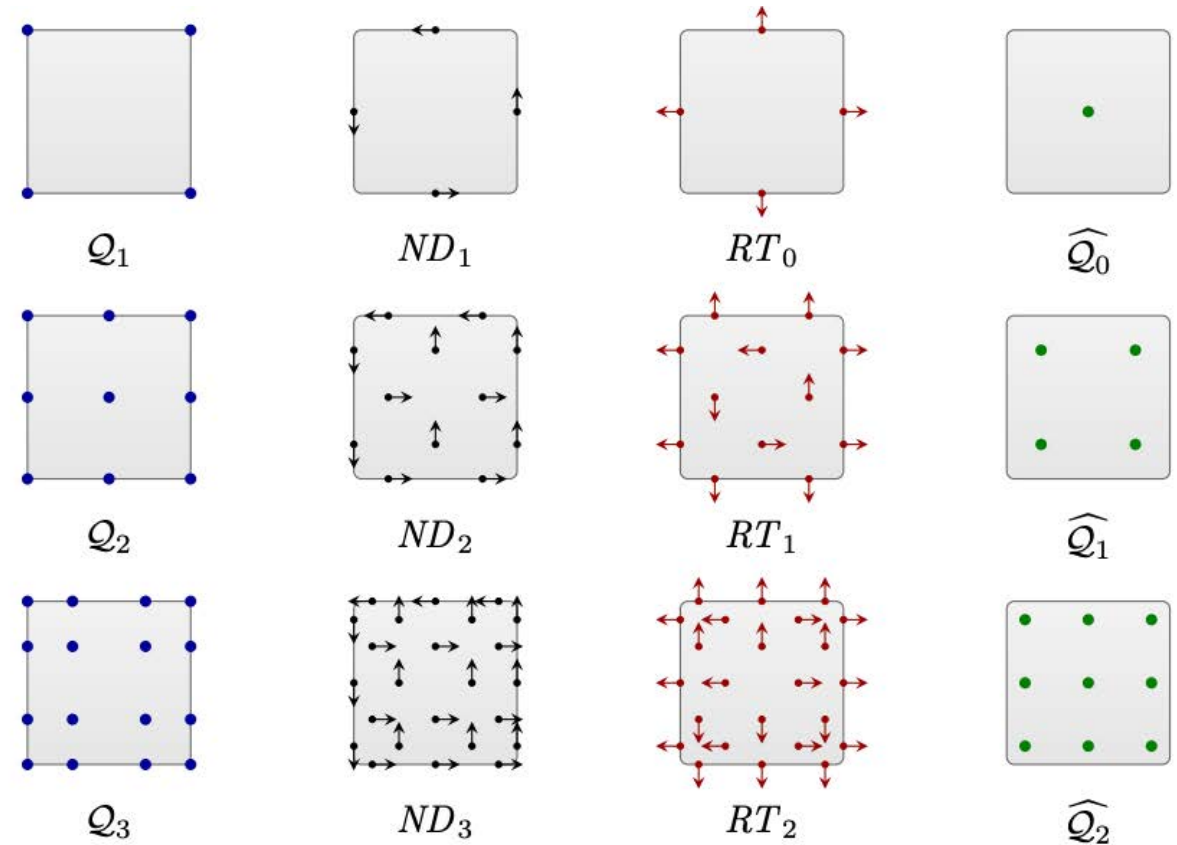
High-order geometry



Lagrangian 8th order example



High-order spaces and degrees of freedom



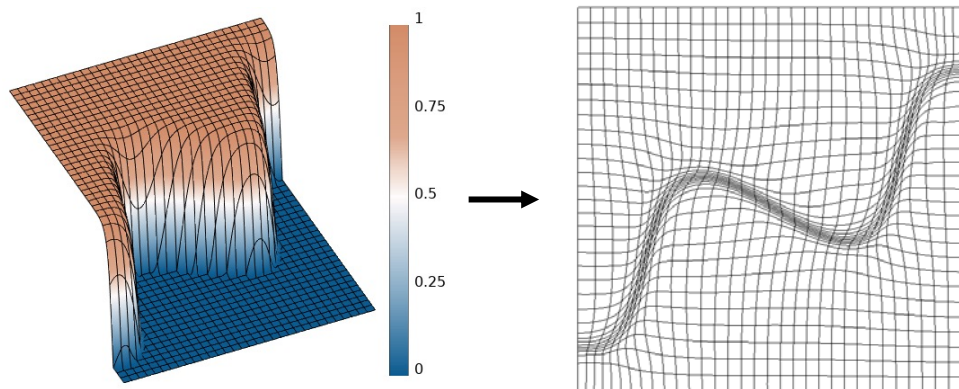
High-Order Mesh Optimization (TMOP)

MFEM provides both geometric and simulation-driven adaptivity

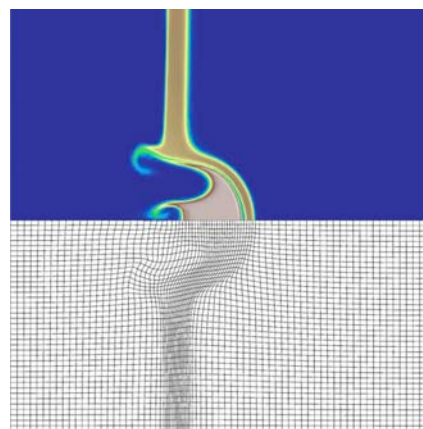


- User-controlled specification of the target mesh
 - Control over: size/skew/aspect ratio/rotation
- Variational minimization through FE operations
 - Generality: dimension/element type/mesh order
 - Facilitates matrix-free formulations and GPU porting

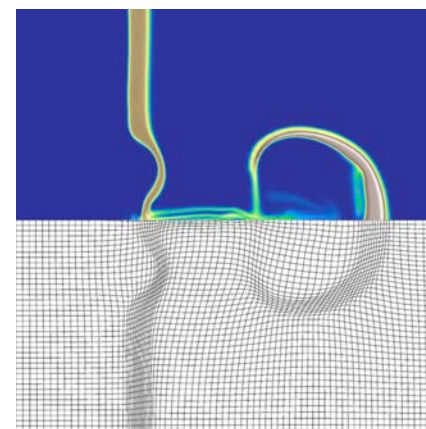
Minimizes $\sum_{E \in \mathcal{M}} \int_{E_t} \mu(T(x_t))$ mesh quality metric computed at q-points



Adaptivity of shape and size to a discrete interface



Adaptivity in a high-velocity impact simulation in MARBL



IMR Best Paper

Matrix-free Preconditioning

Optimal memory and near-optimal computations

Memory usage and comput. complexity: scaling with p		3D
Number of DOFs	$\mathcal{O}(p^d)$	p^3
Matrix-based methods		
Nonzeros in system matrix	$\mathcal{O}(p^{2d})$	p^6
Traditional (naïve) assembly	$\mathcal{O}(p^{3d})$ ops	p^9
Sum-factorized assembly	$\mathcal{O}(p^{2d+1})$ ops	p^7
“Matrix-free” methods		
Optimal memory usage	$\mathcal{O}(p^d)$	p^3
Sum-factorized operator application	$\mathcal{O}(p^{d+1})$ ops	p^4

Goal: Iterative solvers with:

optimal $\mathcal{O}(p^d)$ memory

$\mathcal{O}(p^{d+1})$ operations

$\mathcal{O}(1)$ iterations

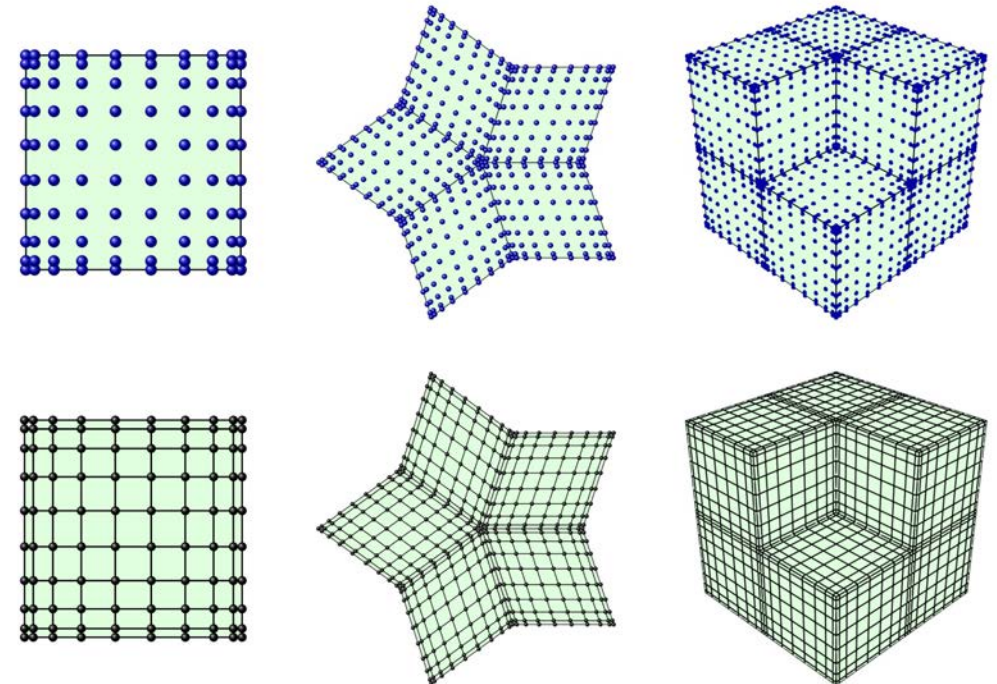
⇒ Cannot assemble the matrix

⇒ Must construct preconditioners without access to matrix entries

Main preconditioning approaches

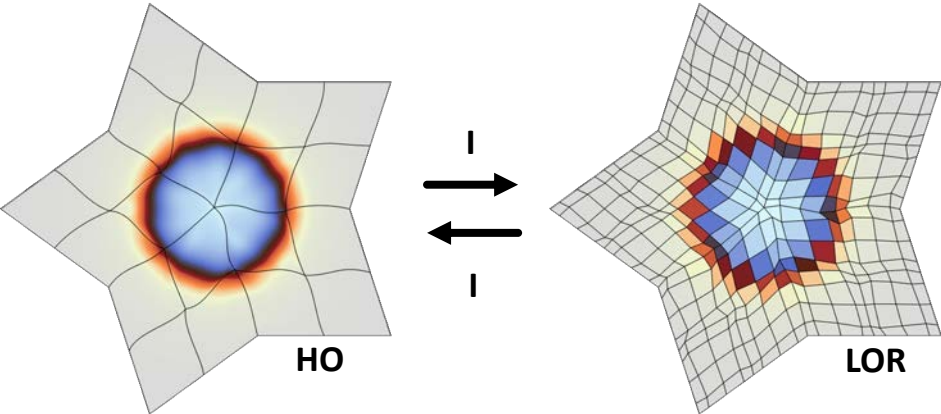
- p-Multigrid + FEM-SEM
- Orszag (1980), Canuto and Quarteroni (1985, 1994, 2010,...), Deville and Mund (1985, 1990), Fischer (1997), Munz (2014), Vilar (2019)

Low-order refined (LOR)



Low-Order-Refined Solvers

Matrix-free preconditioners for PA operators

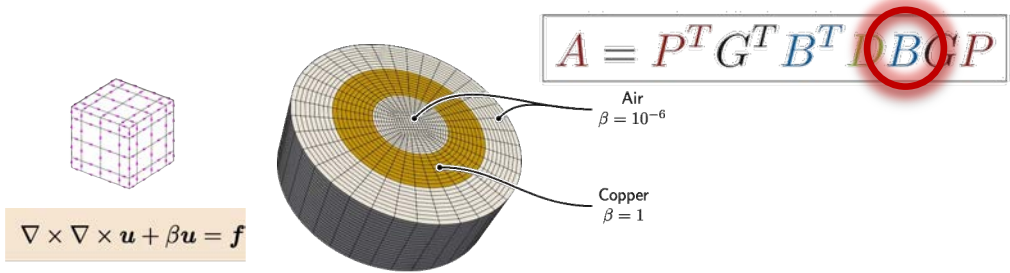


- Pick LOR space and HO basis so $\mathbf{P}=\mathbf{R}=\mathbf{I}$ (Gerritsma, Dohrmann)
- $\mathbf{A}_{\text{LOR}}$ is sparse and spectrally equivalent to $\mathbf{A}_{\text{HO}}$

Theorem 2. Let $M_\star$ and $K_\star$ denote the mass and stiffness matrices, respectively, where $\star$ represents one of the above-defined finite element spaces with basis as in Section 4.3. Then we have the following spectral equivalences, independent of mesh size h and polynomial degree p .

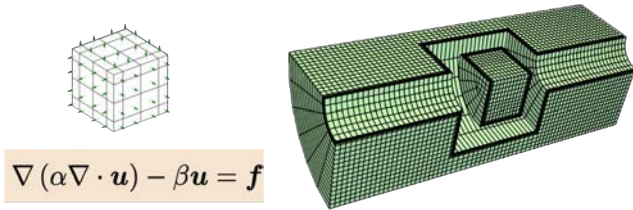
$$\begin{aligned} M_{V_h} &\sim M_{V_p}, & K_{V_h} &\sim K_{V_p}, \\ M_{W_h} &\sim M_{W_p}, & K_{W_h} &\sim K_{W_p}, \\ M_{X_h} &\sim M_{X_p}, & K_{X_h} &\sim K_{X_p}, \\ M_{Y_h} &\sim M_{Y_{p-1}}, \\ M_{Z_h} &\sim M_{Z_p}, & K_{Z_h} &\sim K_{Z_p}. \end{aligned}$$

- $(\mathbf{A}_{\text{HO}})^{-1} \approx (\mathbf{A}_{\text{LOR}})^{-1} \approx \mathbf{B}_{\text{LOR}}$ - can use AMG, AMS, ADS



LOR-AMS						
p	Its.	Assembly (s)	AMG Setup (s)	Solve (s)	# DOFs	# NNZ
2	41	0.082	0.277	0.768	516,820	1.65×10^7
3	63	0.251	0.512	2.754	1,731,408	5.64×10^7
4	75	0.679	1.133	7.304	4,088,888	1.34×10^8
5	62	1.574	2.185	11.783	7,968,340	2.61×10^8
6	89	3.336	4.024	30.702	13,748,844	4.51×10^8

Matrix-Based AMS						
p	Its.	Assembly (s)	AMG Setup (s)	Solve (s)	# DOFs	# NNZ
2	39	0.140	0.385	1.423	516,820	5.24×10^7
3	44	1.368	1.572	9.723	1,731,408	4.01×10^8
4	49	9.668	5.824	45.277	4,088,888	1.80×10^9
5	53	61.726	15.695	148.757	7,968,340	5.92×10^9
6	56	502.607	40.128	424.100	13,748,844	1.59×10^{10}

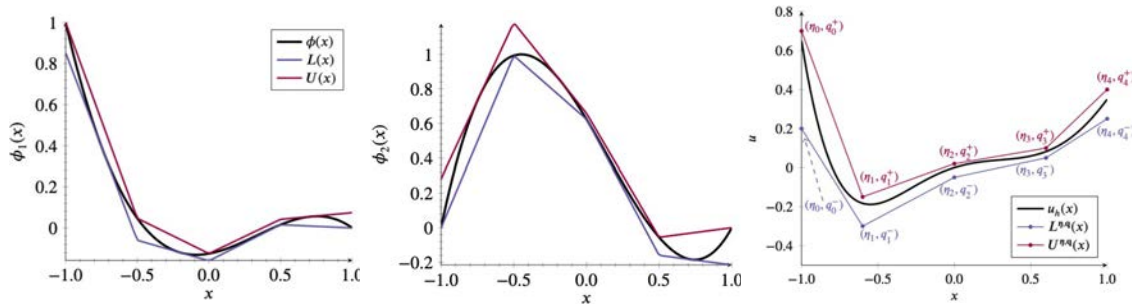


p	LOR-ADS		Matrix-Based ADS		Speedup
	Runtime (s)	Memory (GB)	Runtime (s)	Memory (GB)	
2	2.11	0.04	2.98	0.20	$1.41 \times$
3	6.64	0.15	22.58	1.84	$3.40 \times$
4	17.40	0.35	114.35	9.13	$6.57 \times$
5	43.70	0.68	422.74	32.21	$9.67 \times$
6	92.76	1.18	1324.94	91.09	$14.28 \times$

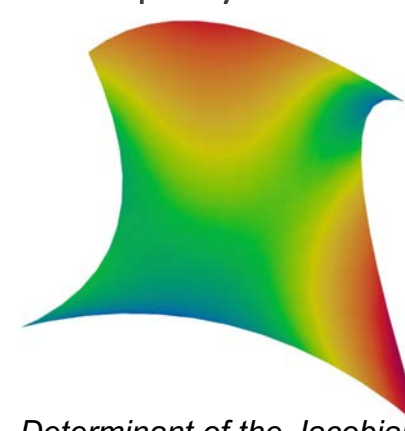
Bounding High-Order FEM Functions

Piecewise linear bounds on high-order multi-dimensional polynomials

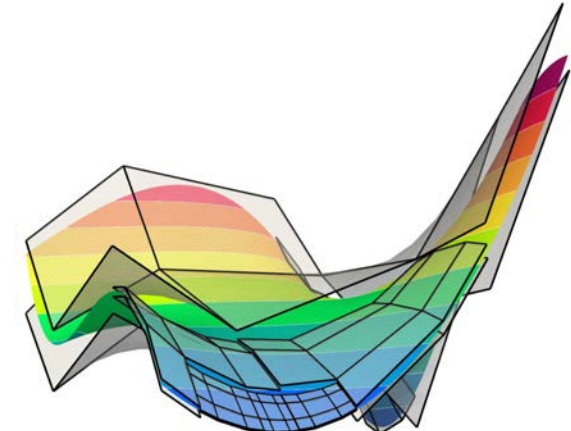
- Global minima and maxima of high-order functions
- Piecewise linear bounds** of the basis functions are used to bound their weighted combination.



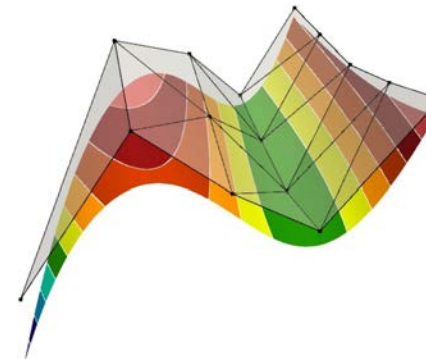
- Computationally cheap**
 - Bounds are computed once
- Provides much **tighter bounds** than convex-hull type approaches (e.g., Bernstein coefficients).
- Method extends to **nodal** and **non-nodal bases** on different **element types** in 2D and 3D



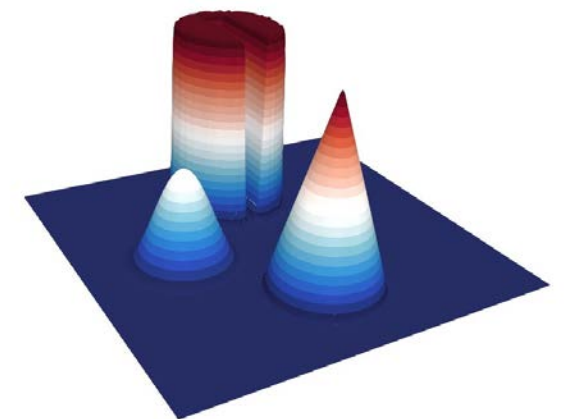
*Determinant of the Jacobian
of the transformation
($p_{\text{mesh}} = 2$, $p_{\text{det}} = 3$)*



*Piecewise linear bounds +
recursion to determine validity*



*Piecewise linear bounds on
simplices*



Advection with bounds preserving limiter

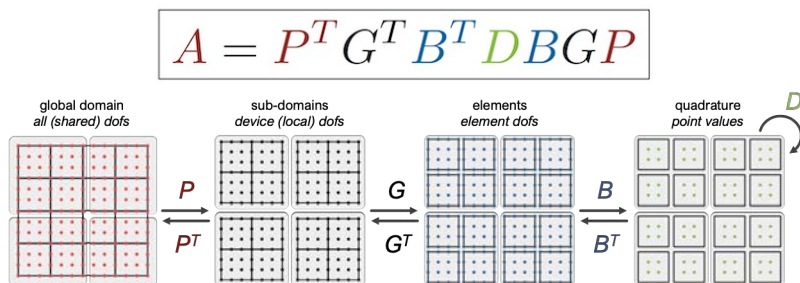
∂FEM: Autodiff for Partially Assembled Operators

Jacobians and derivatives of FEM operators in a user-friendly way

$$A = P^T G^T B^T \textcolor{red}{D} B G P$$



- FEM decomposition



- Parameters $\hat{\rho} = B_{\rho} G_{\rho} P_{\rho} \rho$
- Parametric nonlinear operator

$$A(u; \rho) = P^T G^T B^T D(\hat{u}, \hat{\rho})$$

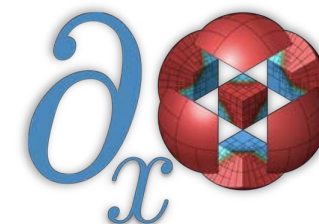
- Need to differentiate at **Q-points** only!

$$\nabla_u A(u; \rho) = P^T G^T B^T \nabla_{\hat{u}} D(\hat{u}, \hat{\rho}) B G P$$

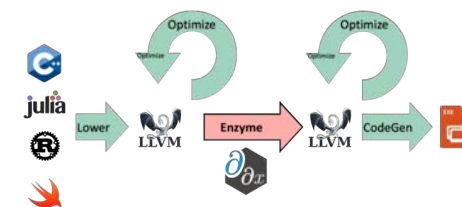
(Jacobian is FEM decomposed linear operator)

- Differentiate the Q-function **D** with Enzyme!

- AD at LLVM level, *after* compiler optimization
- Can mix code from different languages
- Differentiate across function calls (e.g. EOS)
- AD with minimal code changes
- Differentiate only what is necessary



MFEM + Enzyme

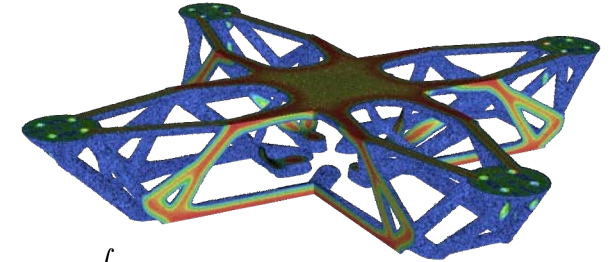


Performing AD at low-level lets us work on optimized code!

Roadmap for Next Year

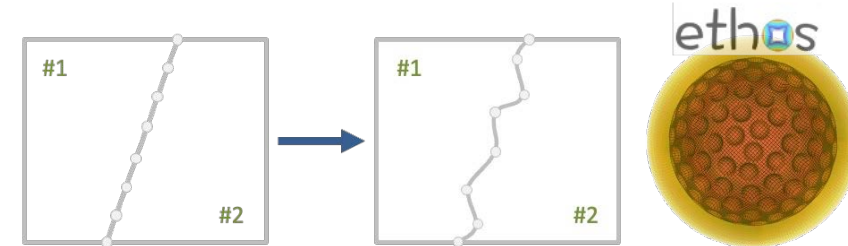
Plans for FY27

- **GPU computing**
 - Kernels using tensor/matrix cores
 - Mixed precision algorithms
 - Efficient high-order methods on mixed meshes · including simplices
- **Differentiable Simulations**
 - Additional dFEM features in next release · support for coupled simulations
 - More examples, miniapps and documentation
- **Adapt to AI changes**
 - Meshing and discretizations for AI workflows
 - Use AI to improve MFEM · make MFEM more useful for AI tools and agents
- **New releases**
 - glvis-4.6 in Dec · open to feature requests
 - mfem-5.0 in Jun · code modernization, API fixes · expect breaking changes
- **What would you like to see?**
 - Slack: [#meet-the-team](#) · GitHub: github.com/mfem/mfem/issues · Email: mfem@llnl.gov

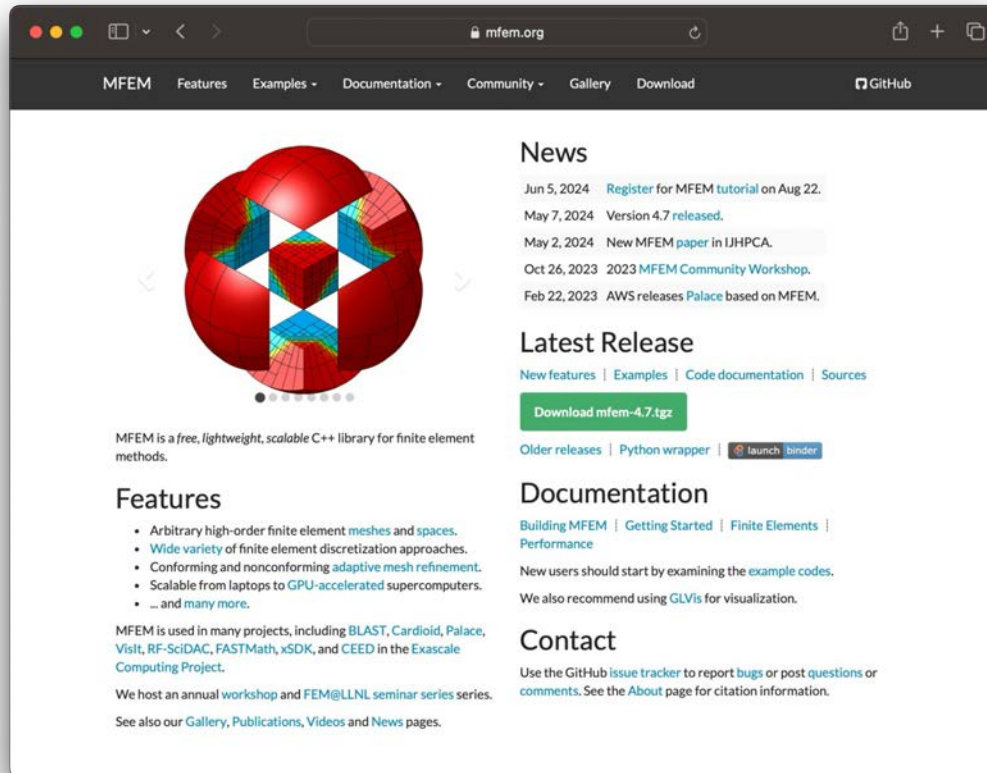


$$\langle F_D(u, \rho), v \rangle = \int_{\Omega} D(u, \nabla u, \rho) \cdot (v, \nabla v)$$
$$\updownarrow \text{ (with a small red cube icon) }$$
$$F_D(u, \rho) = T_v^T D_{\omega}(T_u u, T_{\rho} \rho) \longrightarrow \frac{dF}{du}(u^*, \rho^*) = T_v^T \partial_{\tilde{u}} D_{\omega}(\tilde{u}^*, \tilde{\rho}^*) T_u$$





MFEM Resources



Website:

mfem.org

Software:

github.com/mfem

Publications:

mfem.org/publications

Email:

mfem@llnl.gov

- Contact us with questions + feedback
- Contribute to the code
- Explore our publications

Thank you from the MFEM team at LLNL!



mfem.org



**Lawrence Livermore
National Laboratory**

Disclaimer

This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.